

UNIVERSIDAD TÉCNICA DE AMBATO

Carrera de Ingeniería en Software

Asignatura: Algoritmos de Programación



PROYECTO FORMATIVO INTEGRADOR

LogicWeb UTA: Desarrollo de un aplicativo web educativo para la retroalimentación del aprendizaje de Algoritmos y Lógica de Programación mediante ejercicios interactivos aplicados a problemas de la vida real.

Docente	José Rubén Caiza C.
Asignatura	Algoritmos de Programación
Proyecto	Aplicativo web didáctico de lógica y algoritmos
Curso / paralelo	1ero Software B
Estudiantes	Atiencia Josue, Barrionuevo Job, Rocha Carolina, Segovia Joseph
Periodo académico	2026
Lugar	Ambato – Ecuador
Fecha	17 de Abril de 2026

LogicWeb UTA

Aprender lógica programando soluciones reales

Tabla de contenido

1. RESUMEN EJECUTIVO.....	3
2. INTRODUCCIÓN	4
3. TEMA DEL PROYECTO.....	5
4. TÍTULO DEL PROYECTO.....	5
5. PLANTEAMIENTO DEL PROBLEMA.....	6
6. JUSTIFICACIÓN	6
7.1. Objetivo general	8
7.2. Objetivos específicos	8
9. MARCO TEÓRICO	9
9.1. Lógica de programación.....	9
9.2. Algoritmos	10
9.3. Pseudocódigo y diagramas de flujo	10
9.4. Variables y operadores	14
9.5. Programación orientada a objetos	16
9.6. Estructuras de datos.....	17
9.7. Aplicativos webs educativos.....	17
10. METODOLOGÍA DE DESARROLLO	18
10.1 . Fase de análisis	18
10.2 . Fase de diseño	18
10.3 Fase de implementación.....	18
10.4 Fase de pruebas.....	19
10.5 Fase de presentación	19
11. REQUERIMIENTOS DEL SISTEMA.....	19
11.1. Requerimientos funcionales.....	19
11.2. Requerimientos no funcionales	20
12. ARQUITECTURA PROPUESTA.....	20
13. DISEÑO FUNCIONAL DEL SISTEMA	21
13.1. Módulo de Gestión de Usuarios	21
13.2. Módulo de contenidos teóricos	21
13.3. Módulo de ejercicios resueltos	21
13.4. Módulo de ejercicios interactivos	22
13.5. Módulo de retroalimentación	22
13.6. Módulo de Reportes y Estadísticas	22

14. DISEÑO DE DATOS.....	22
14.1. Entidades principales	22
14.2. Relaciones principales	23
15. INTEGRACIÓN DE CONOCIMIENTOS DE LA ASIGNATURA	23
16. PLAN DE TRABAJO	27
17. CRONOGRAMA	30
18. RECURSOS	32
18.1. Recursos Humanos.....	33
a. Equipo Desarrollador.....	33
b. Docente tutor	33
c. Usuarios Beneficiarios.....	33
18.2. Recursos tecnológicos en Hardware	33
a. Equipos de Cómputo.....	33
b. Infraestructura de Red	33
18.3. Recursos tecnológicos en Software	34
a. Entornos y Herramientas de Desarrollo	34
b. Tecnologías del Backend	34
c. Tecnologías del Frontend	34
d. Motor de Compilación	34
e. Sistema de Gestión de Base de Datos	34
f. Control de Versiones	34
18.4. Recursos Institucionales	34
a. Espacios Físicos	35
b. Fuentes de Información	35
19. PRESUPUESTO ESTIMADO.....	35
19.1. Viabilidad Financiera y Sostenibilidad	36
20.1. Consideraciones generales sobre la gestión de riesgos	39
21. RESULTADOS ESPERADOS	39
21.1. Impacto académico esperado.....	41
21.2. Indicadores de éxito	41
Bibliografía	43

1. RESUMEN EJECUTIVO

El presente proyecto titulado “LogicWeb UTA: Desarrollo de un aplicativo web educativo para la retroalimentación del aprendizaje de Lógica y Algoritmos de Programación mediante ejercicios interactivos aplicados a problemas de la vida real” tiene como finalidad reforzar el aprendizaje de la materia de Algoritmos y Lógica de Programación, para ello se propone el diseño y desarrollo de un aplicativo web educativo. La solución a este problema es integrar contenidos teóricos como prácticos, con el fin de poder mejorar la comprensión de la asignatura, la práctica y la retroalimentación a nuestros estudiantes interesados en la informática.

La iniciativa busca responder a la demanda de estudiantes interesados en integrarse al entorno del Software fomentando al uso de aplicativos webs como medio didáctico de enseñanza. Para ello se lo ha organizado en módulos que incluyen teoría resumida, ejercicios resueltos, ejercicios prácticos, corrección a errores, recomendaciones al usuario y estadísticas de progreso. Se dará uso a tecnologías accesibles, Flask en Python para la gestión de la web, C++ para el desarrollo de los algoritmos, HTML 5, CSS 3, Bootstrap y JavaScript para la interfaz gráfica y finalmente SQLite para la base de datos.

El Proyecto busca transformar los conceptos aprendidos en clase en experiencias de aprendizaje y relacionadas al contexto universitario, como la aplicación en la vida real de conocimientos adquiridos. Al implementar teoría, práctica y retroalimentación en nuestro aplicativo se convierte en un recurso didáctico para ser utilizado en clases, prácticas y procesos de nivelación o refuerzo.

2. INTRODUCCIÓN

La formación inicial en programación exige que el estudiante desarrolle una base sólida de pensamiento lógico, análisis de problemas y diseño de soluciones. Sin embargo, en muchos contextos académicos se evidencia un obstáculo entre la comprensión teórica de los contenidos y su aplicación práctica en escenarios reales. Esta dificultad se refleja en errores frecuentes al definir entradas, procesos y salidas, al plantear algoritmos y al implementar soluciones correctas en cualquier lenguaje de programación.

Ante esta necesidad, el proyecto LogicWeb UTA – Soluciones reales plantea el desarrollo de un aplicativo web educativo utilizado como recurso de apoyo para las clases de Algoritmos y Lógica de Programación. El sistema estará enfocado en la retroalimentación del aprendizaje a través de ejemplos, ejercicios y prácticas interactivas que permitan reforzar la lógica de programación de forma progresiva.

La presente propuesta implementa herramientas gratuitas como Flask para gestionar el aplicativo, C++ como un lenguaje el cual nos permite realizar la ejecución de algoritmos prácticos, también el uso de HTML, CSS, Bootstrap y JavaScript para la interfaz y SQLite para gestionar datos. Mediante las tecnologías mencionadas se busca aplicar los conocimientos dados por el docente de forma práctica y experimental. Además, se basa en problemas de la vida real haciendo que este contribuya a que los estudiantes reconozcan la importancia de la programación en la vida académica y profesional.

3. TEMA DEL PROYECTO

Desarrollo de un aplicativo web educativo para reforzar el aprendizaje de lógica y algoritmos mediante ejercicios interactivos y prácticos en C++.

4. TÍTULO DEL PROYECTO

LogicWeb UTA: Desarrollo de un aplicativo web educativo para la retroalimentación del aprendizaje de Algoritmos y Lógica de Programación mediante ejercicios interactivos aplicados a problemas de la vida real.

5. PLANTEAMIENTO DEL PROBLEMA

En el proceso de la formación universitaria como ingenieros en Software, la materia de Algoritmos y Lógica de programación forma un pilar importante para desarrollar el análisis y razonamiento para dar solución a problemas informáticos. Sin embargo, se puede observar que la mayoría de estudiantes de la Universidad Técnica de Ambato tienen ciertas dificultades a la hora de aplicar conocimientos teóricos en prácticas, lo que significa la aparición de errores frecuentes al momento de la codificación de programas, aplicaciones o estructuras de control y datos.

La enseñanza tradicional hoy en día se basa en ejercicios hechos a papel, y esto impide la retroalimentación y la interacción del estudiante con problemas relacionados a la vida cotidiana. Esto produce que la teoría aprendida no se complemente con la práctica, afectando al rendimiento académico de los estudiantes.

Frente a esta realidad, surge la necesidad de construir una herramienta académica que integre teoría, práctica y seguimiento del aprendizaje. El proyecto propone un aplicativo web didáctico capaz de presentar contenidos de forma estructurada y permitir la resolución de ejercicios con orientación y retroalimentación dentro de un entorno amigable.

Por lo tanto, la solución central que busca brindar el presente proyecto es facilitar la comprensión y aplicación de la lógica y los algoritmos de la programación mediante ejercicios basados en problemas reales, añadiendo tecnologías accesibles, atractivas, pero sobre todo manejables para nuestros estudiantes de la Universidad Técnica de Ambato o para terceros interesados en aprender sobre software y su desarrollo.

6. JUSTIFICACIÓN

La enseñanza de la programación en los primeros niveles suele representar un desafío para los estudiantes los cuales deben desarrollar habilidades analíticas, razonamiento y pensamiento lógico. Sin embargo, la enseñanza tradicional se enfoca en la exposición teórica y en ejercicios limitados, esto no garantiza la comprensión de conceptos generando dificultades en el cambio de lo teórico hacia lo práctico.

Ante esta situación, el desarrollo del proyecto LogicWeb UTA – Soluciones reales se justifica ofreciendo un entorno educativo intuitivo y fácil de manejar, integrando teoría, ejemplos resueltos y ejercicios prácticos, ayudando al estudiante a comprender de mejor manera la programación y aplicar sus conocimientos teóricos en la práctica de forma manejable y entendible reforzando su aprendizaje o adquirirlo. También, al basarse en problemas cotidianos, el sistema contribuirá a que los estudiantes puedan reconocer lo importante que es la programación para la vida actual.

Además, el uso de tecnologías accesibles y gratuitas como Flask con Python, C++, HTML, CSS, Bootstrap, JavaScript y SQLite garantiza que nuestro proyecto sea viable y se pueda adaptar a diferentes contextos académicos sin el requerimiento de grandes inversiones. Esto lo puede convertir en una herramienta innovadora en su método de enseñanza.

Finalmente, el proyecto aporta un valor significativo en el crecimiento formativo de los usuarios interesados en este programa y al aprendizaje autónomo. También, solventa las necesidades actuales de aprendizaje usando tecnología la cual es indispensable para el mundo actual además de ser más atractiva y práctica que la educación convencional.

7. OBJETIVOS

7.1. Objetivo general

Desarrollar un aplicativo web educativo llamado LogicWeb UTA – Soluciones Reales, que permita brindar un refuerzo del aprendizaje acerca de Lógica y Algoritmos de Programación mediante el uso de contenidos teóricos, prácticos, y ejemplos enfocados en C++, integrando la retroalimentación de la materia para mejorar la comprensión y aplicación práctica de la programación en la vida cotidiana.

7.2. Objetivos específicos

1. Analizar las necesidades de aprendizaje relacionadas con la asignatura de Lógica y Algoritmos de Programación.
2. Diseñar una interfaz web clara, amigable y orientada al uso académico.
3. Construir el módulo de los contenidos teóricos con explicaciones claras y resumidas.
4. Implementar ejercicios interactivos con validación de entradas, procesamiento y retroalimentación de resultados.
5. Integrar una base de datos en SQLite para el registro de usuarios, almacenamiento de intentos y generación de reportes de progreso académico.
6. Desarrollar ejercicios resueltos en C++ que indiquen paso a paso la resolución de estos mediante pseudocódigos y diagramas de flujo.
7. Evaluar el funcionamiento del aplicativo como recurso de apoyo al aprendizaje.

8. ALCANCE DEL PROYECTO

El proyecto LogicWeb UTA – Soluciones reales, tiene como alcance el diseño y desarrollo de un aplicativo web educativo orientado a reforzar el aprendizaje de Algoritmos y Lógica de Programación que incluya: página principal informativa, organización por temas o unidades, teoría resumida, ejercicios resueltos, ejercicios interactivos, mensajes de retroalimentación y almacenamiento de resultados.

El proyecto se limitará a un entorno académico teniendo un alcance orientado a la enseñanza de la lógica y los algoritmos de programación necesarios para la formación académica. Su objetivo principal es brindar una herramienta de apoyo y refuerzo del aprendizaje inicial de la programación.

9. MARCO TEÓRICO

9.1. Lógica de programación

a. Explicación acerca de la lógica de programación

Comencemos por expresar un punto de vista sobre María. ¿Y quién es Mana? Es la persona que nos guiará en esta explicación. Se lo diré de forma muy sencilla: "Mana es alta". De inmediato, usted, estimado lector, visualizará a una mujer que supera el metro setenta de altura, quien podría ser más alta que usted o al menos de la misma altura (si es que usted se considera alto). En el momento que yo digo "María es alta", usted tendrá que hacer algunas reflexiones lógicas y concluyentes para comprender lo que intenté transmitir. ¿Qué fue lo que describí sobre Mana? Muy sencillo, mencioné una cualidad de ella. ¿Qué es una cualidad? Es una característica que distingue a un ente informático. ¿Y qué es un ente informático? Todo lo que puede ser descrito a partir de sus características. (Trejos Buritica, 2017)

¿Qué rasgos posee un atributo? El primero es que sigue una serie de razonamientos humanos, lo que indica que es necesario contar con un conjunto previo de consideraciones. La segunda rasgo es su naturaleza muy subjetiva. Si Mana mide 1,65 y reside en Occidente, quizás no se la considere alta; sin embargo, si está en Europa, puede ser vista como baja, y si se encuentra en Oriente, sería percibida como notablemente alta. Los atributos siempre estarán influenciados por la perspectiva del que utiliza el término. Un ejemplo similar ocurre cuando un hombre describe a una mujer como muy bella; lo que él considera belleza podría no serlo para otros. (Trejos Buritica, 2017)

Como se observa, estas dos características, junto con un conjunto de conceptos y experiencias arraigadas en la cultura de la región en la que hemos crecido, contribuyen a reforzar la relatividad de un atributo. Debido a esta conceptualización subjetiva de los atributos, estos resultan difíciles de manejar, ya que su interpretación dependerá de la persona que los evalúe. (Trejos Buritica, 2017)

9.2. Algoritmos

Formalmente, un algoritmo se describe como una serie de pasos, procedimientos o acciones que se ejecutan de forma secuencial y detallada con el propósito de resolver un problema o alcanzar un objetivo. Por ejemplo, el acto de comenzar la lectura de un libro se efectúa de manera automática, sin que se tome en cuenta el proceso involucrado para completarlo. Los pasos más elementales para leer un libro incluirían: abrirlo en la primera página, leer la página, voltear la hoja, leer la página siguiente y continuar de esta manera hasta finalizar la lectura del libro y cerrarlo. (Gaxiola Pcheco, 2008)

9.3. Pseudocódigo y diagramas de flujo

a. Pseudocódigo

El pseudocódigo constituye un idioma no formal y artificial que asiste a los programadores en la creación de algoritmos. Este se asemeja al lenguaje que usamos en nuestra vida diaria; es accesible y fácil de entender para el usuario, aunque no es un auténtico lenguaje de programación. No se ejecuta en dispositivos informáticos, sino que facilita al desarrollador el proceso de pensar en un programa antes de intentar plasmarlo en algún lenguaje de programación. Un programa que se ha traducido a pseudocódigo puede ser fácilmente transformado en un programa en C/C++, siempre que esté bien diseñado. (Gaxiola Pcheco, 2008)

b. Diagrama de flujo

Un esquema de flujo se describe como la ilustración visual de un algoritmo. La creación de un esquema de flujo es necesaria ya que, en general, es más sencillo captar la información a través de imágenes que mediante un escrito, especialmente cuando el algoritmo se extiende en tamaño. En un esquema de flujo se incluyen formas que contienen datos, además de flechas que señalan el orden de los pasos. A estas formas se les denominará a partir de ahora bloques; hay diversos tipos de bloques: (Gaxiola Pcheco, 2008)

c. Bloques Terminales

Se emplean para indicar el principio y el fin del diagrama de flujo. Su forma es:

Figura 1

Bloque terminal de inicio y fin



Gaxiola Pacheco, C. G. (2008). Ejemplo de pseudocódigo en C [Imagen]. En Metodología de la programación con pseudocódigo enfocado al lenguaje C (p. 32). eLibro.

<https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>

d. Bloques de entrada/salida

Se emplean para introducir datos a la computadora o exteriorizarlos. El bloque de entrada tiene la siguiente forma:

Figura 2

Bloque terminal de entrada

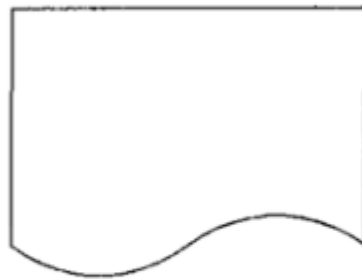


Gaxiola Pacheco, C. G. (2008). Ejemplo de pseudocódigo en C [Imagen]. En Metodología de la programación con pseudocódigo enfocado al lenguaje C (p. 33). eLibro. <https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>

El bloque de salida tiene la siguiente forma:

Figura 3

Bloque terminal de salida



Gaxiola Pacheco, C. G. (2008). Ejemplo de pseudocódigo en C [Imagen]. En Metodología de la programación con pseudocódigo enfocado al lenguaje C (p. 33). eLibro. <https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>

e. Bloques de proceso

En ellos se indica una acción imperativa o una operación aritmética. Su forma es:

Figura 4

Bloque terminal de procesos



Gaxiola Pacheco, C. G. (2008). Ejemplo de pseudocódigo en C [Imagen]. En Metodología de la programación con pseudocódigo enfocado al lenguaje C (p. 33). eLibro.

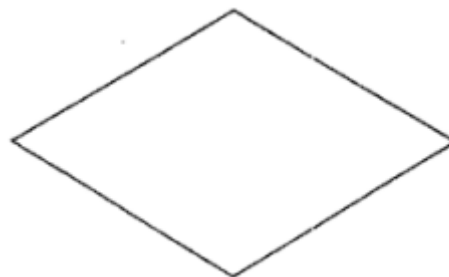
<https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>

f. Bloque condicional

Son preguntas del tipo SI O NO, que pueden alterar la secuencia en la ejecución de los bloques del diagrama de flujo, su forma es:

Figura 5

Bloque terminal de condición



Gaxiola Pacheco, C. G. (2008). Ejemplo de pseudocódigo en C [Imagen]. En Metodología de la programación con pseudocódigo enfocado al lenguaje C (p. 34). eLibro.

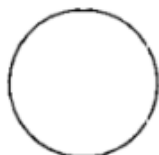
<https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>

g. Bloques de conexión

Son figuras que determinan que la secuencia de acciones en un diagrama continúa en alguna otra parte Su forma es:

Figura 6

Bloque terminal de conexión



Gaxiola Pacheco, C. G. (2008). Ejemplo de pseudocódigo en C [Imagen]. En Metodología de la programación con pseudocódigo enfocado al lenguaje C (p. 34). eLibro.

<https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>

9.4. Variables y operadores

a. Variables

Son áreas de almacenamiento que modifican su contenido a lo largo del proceso de tratamiento de la información. (Gaxiola Pacheco, 2008)

b. Operadores

Una expresión se forma por medio de operadores y operandos. Un operador representa un signo o término que indica qué tipo de acción se llevará a cabo entre uno o dos valores, que son conocidos como operandos. Estos pueden ser tanto constantes como variables. Se reconocen cuatro categorías de operadores: aritméticos, relacionales y lógicos: (Gaxiola Pacheco, 2008)

c. Aritméticos

Figura 7

Operador aritmético

<i>Operador</i>	<i>operación</i>
$\wedge$	Potenciación
$*, /$	Multiplicación, División
$+, -$	Suma, Resta

d. Relacionales

Figura 8

Operador relacional

<i>Operador</i>	<i>operación</i>
$=$	Igual que
$< > \neq > <$	Diferente que

Figura 9

Operador relacional

<	Menor que
>	Mayor que
< =	Menor o igual que
> =	Mayor o igual que

e. Lógicos

Figura 10

Operador lógico

<i>Operador</i>	<i>operación</i>
NOT	Negación
AND	Conjunción
OR	Disyunción

9.5. Programación orientada a objetos

La programación basada en objetos constituye un enfoque de programación que se aparta del método tradicional, ya que emplea objetos y sus acciones para abordar problemas y crear software. (Moreno, 2015)

Con la programación basada en objetos (Poo), se mejora la modularidad del software y su capacidad de reutilización. Asimismo, la Poo se distingue de la programación tradicional al emplear técnicas innovadoras como el polimorfismo, la encapsulación y la herencia, entre otras. (Moreno, 2015)

Habitualmente, los lenguajes modernos de programación facilitan tanto la programación basada en objetos como la programación tradicional. Por lo tanto, se puede considerar la Poo

como una progresión de la programación tradicional (programación estructurada). (Moreno, 2015)

9.6. Estructuras de datos

Las variables que pertenecen a un tipo compuesto pueden tener múltiples valores simultáneamente en un instante específico. Estas variables simbolizan grupos o colecciones de valores. Los tipos compuestos también reciben el nombre de “tipos estructurados” o “estructuras de datos”. (Fritelli & Guzman, 2020)

Las estructuras de datos son esenciales para la creación de programas que requieren gestionar grandes cantidades de información o resultados, o cuando la forma en que se agrupan dichos datos o resultados es compleja. El estudiante debería considerar lo difícil que sería desarrollar un programa que administre la información de todos los estudiantes de la Facultad, utilizando únicamente variables sencillas, como se ha hecho hasta ahora. Las estructuras de datos hacen posible la organización de toda la información necesaria para un programa de manera eficiente y ordenada, además de facilitar el acceso individual a cada uno de esos datos. (Fritelli & Guzman, 2020)

El conjunto de estructuras de datos que un lenguaje proporciona se llama "estructuras nativas de datos del idioma". Así, los arreglos, registros y archivos provienen de las estructuras incorporadas del lenguaje C/C++. Sin embargo, a nivel teórico, muchas más estructuras de datos pueden ser definidas que las ofertas de un lenguaje específico. Las estructuras teóricas (o "abstractas") que podrían ser útiles en un programa, pero no son soportadas por el lenguaje generalmente se llaman "estructuras de datos no nativas" para ese idioma. Algunas estructuras no nativas en el lenguaje C/C++ incluyen baterías, colas, listas, árboles y gráficos. Algunos de ellos serán examinados más adelante en lo que queda de este tema. (Fritelli & Guzman, 2020)

9.7. Aplicativos webs educativos

El término "aplicación educativa" abarca cualquier programa, recurso o contenido multimedia que se quiera utilizar en dispositivos electrónicos. También se considera una herramienta tecnológica útil en la educación. (Universidad Europea, 2023)

Las aplicaciones educativas móviles tienen como objetivo ayudar con el aprendizaje mediante el uso de tecnología educativa. En otras palabras, su objetivo es crear aplicaciones sencillas, honestas y divertidas para que los niños puedan aprender más feliz y eficazmente. (Universidad Europea, 2023)

10. METODOLOGÍA DE DESARROLLO

10.1. Fase de análisis

En esta fase se identificará las necesidades académicas de los estudiantes en la materia de Algoritmos y Lógica de Programación. Se definirán los requerimientos para realizar el aplicativo web, los módulos que conformarán el proyecto y los problemas de la vida cotidiana se la integrarán como ejercicios prácticos en C++. El análisis ayuda a establecer las bases teóricas y técnicas de nuestro proyecto.

10.2. Fase de diseño

En este apartado se establecerá la arquitectura que tendrá el proyecto, detallando el frontend, backend y su base de datos. Se elaborará prototipos de interfaz en Figma para poder garantizar el manejo intuitivo del aplicativo web y se modelará también la base de datos en SQLite. También, se diseñará diagramas de flujo y pseudocódigos que servirán como guía para el planteamiento de los ejercicios correspondientes.

10.3 Fase de implementación

Aquí se construyen los módulos que tendrá el aplicativo web. El Frontend se desarrolla en HTML 5, CSS 3, Bootstrap y JavaScript y el backend se lo implementará en Flask con Python. Los ejercicios y algoritmos se los codifica con lenguaje C++ usando Visual Studio Code y finalmente se los integrará con Flask. La base de datos se la realiza con SQLite para poder registrar y guardar usuarios y sus respectivas estadísticas.

10.4 Fase de pruebas

En esta etapa se realiza pruebas de funcionalidad y compatibilidad para poder verificar la correcta ejecución de los módulos de nuestro aplicativo. También se lleva a cabo pruebas de manejo del proyecto por los usuarios permitiendo así detectar errores para posteriormente darles el correcto arreglo o mejoras en caso de que se las necesiten para poder elevar la productividad de esta.

10.5 Fase de presentación

En la última fase, se lleva a cabo la finalización del documento completo del proyecto, incluyendo un manual de usuario y su respectiva guía técnica dirigida a los usuarios. También, se presentarán los resultados obtenidos y se los expondrá mediante diapositivas ante los estudiantes. Además, se tomará en cuenta las recomendaciones de los usuarios para fortalecer la enseñanza hacia la gente interesada en el mundo virtual.

11. REQUERIMIENTOS DEL SISTEMA

11.1. Requerimientos funcionales

- Mostrar una página principal informativa con acceso a los módulos del sistema.
- Organizar contenidos por unidades o temas de la asignatura.
- Presentar teoría resumida de lógica y algoritmos.

- Mostrar ejercicios resueltos paso a paso.
- Permitir el ingreso de datos en formularios interactivos.
- Procesar ejercicios aplicados a casos de la vida real.
- Validar entradas del usuario y controlar errores.
- Generar resultados y mensajes de retroalimentación.
- Guardar información de ejercicios, usuarios o intentos.
- Mostrar reportes básicos o historial de práctica.

11.2. Requerimientos no funcionales

- Interfaz clara, amigable y ordenada.
- Diseño adaptable a diferentes resoluciones de pantalla.
- Tiempos de respuesta adecuados.
- Código modular y documentado.
- Navegación sencilla y comprensible.
- Mensajes de error entendibles para el usuario.
- Mantenimiento y ampliación futura viables.

12. ARQUITECTURA PROPUESTA

Se propone una arquitectura web de tres capas, adecuada para proyectos académicos escalables y fáciles de mantener.

Tabla 1

Estructura del proyecto web

Capa	Tecnologías	Responsabilidad
------	-------------	-----------------

Presentación (Frontend)	HTML, CSS, Bootstrap, JavaScript	Mostrar los contenidos teóricos, formularios y la interfaz gráfica.
Funcionamiento del programa (Backend)	Python (Flask)	Reglas del sistema, cálculos, validaciones y servicios.
Ejercicios Algorítmicos	C++	Resolver problemas, recibir entradas desde formularios y devolver los resultados al backend.
Datos (Base de datos)	SQLite	Almacenar usuarios registrados, estadísticas y progresos.

13. DISEÑO FUNCIONAL DEL SISTEMA

13.1. Módulo de Gestión de Usuarios

Este módulo se encarga de registrar y autenticar a los usuarios permitiendo que el acceso al aplicativo sea seguro. Además, gestiona la información de los usuarios y controla los permisos para poder ingresar a cada uno de los apartados del sistema.

13.2. Módulo de contenidos teóricos

Aquí se dan las explicaciones resumidas sobre lógica, algoritmos, pseudocódigo y diagramas de flujo. Su función se basa principalmente en servir como material didáctico y de apoyo a los estudiantes, facilitando el aprendizaje del usuario en la comprensión de la programación en un lenguaje determinado, es decir, C++.

13.3. Módulo de ejercicios resueltos

Este módulo brinda ejemplos prácticos detallados paso a paso, incluyendo su respectivo pseudocódigo, diagramas de flujo y soluciones en el lenguaje de C++. Su propósito es que los

estudiantes apliquen de forma correcta cada uno de los conceptos teóricos mediante casos concretos y usuales del día a día.

13.4. Módulo de ejercicios interactivos

En este apartado, el estudiante debe ingresar sus datos en formularios web que pasarán por el backend en Flask con Python, y los programas realizados en C++ se ejecutan con estas entradas y devolverán resultados, permitiendo la práctica directa con el uso de algoritmos.

13.5. Módulo de retroalimentación

Analizará los datos obtenidos en cada uno de los ejercicios planteados generando mensajes claros sobre el rendimiento del usuario mostrando sus aciertos y errores. Además, por cada uno de los errores cometidos se mostrarán recomendaciones para poder reforzar las áreas donde el estudiante tiene debilidades.

13.6. Módulo de Reportes y Estadísticas

Finalmente, en este módulo se almacenan los intentos y resultados obtenidos en la base de datos SQLite generando reportes sobre el progreso de aprendizaje académico. El objetivo de esta es mostrar los avances, identificar deficiencias y brindar un seguimiento del aprendizaje de cada estudiante.

14. DISEÑO DE DATOS

El sistema de LogicWeb UTA – Soluciones Reales utiliza base de datos hecha en SQLite, diseñada para ser ligera y lo suficiente para almacenar información relevante académica de los usuarios. El diseño de estos datos se centra en brindar disponibilidad de la información.

14.1. Entidades principales

a. Entidad Usuarios

En la entidad usuarios se almacena la información necesaria de cada estudiante incluyendo un identificador único, Su función es importante ya que se enfoca en gestionar el registro y autenticación.

b. Entidad Ejercicios

Esta contiene cada uno de los problemas de lógica y algoritmos de programación. Cada uno de los registros tiene un identificador, la descripción de su problema, pseudocódigo, diagrama de flujo y el archivo en C++ permitiendo relacionar los contenidos al momento de la práctica.

c. Entidades Intentos

La entidad intentos guardar cada ejecución realizada por los estudiantes en cada uno de los ejercicios prácticos planteados. Se almacena el identificador, el ejercicio realizado, los datos ingresados y los resultados obtenidos permitiendo así llevar un historial del desempeño académico del usuario.

14.2. Relaciones principales

- Un tema puede tener varios ejercicios.
- Un usuario puede registrar múltiples intentos.
- Un ejercicio puede tener varios intentos asociados.
- Cada ejercicio puede disponer de mensajes de retroalimentación.

15. INTEGRACIÓN DE CONOCIMIENTOS DE LA ASIGNATURA

El proyecto LogicWeb UTA – Soluciones Reales constituye una síntesis aplicada de los principales contenidos desarrollados a lo largo de la asignatura de Algoritmos y Lógica de Programación. A continuación, se detalla la forma en que cada unidad temática se articula con el sistema propuesto:

15.1. Lógica de programación

La lógica de programación representa el fundamento conceptual sobre el cual se estructuran todos los módulos del aplicativo. El sistema está diseñado para que el estudiante desarrolle progresivamente su capacidad de razonamiento lógico a través de la resolución de ejercicios interactivos. Cada problema presentado en la plataforma exige que el usuario identifique entradas, defina un proceso y determine la salida esperada, reproduciendo fielmente el esquema del pensamiento lógico computacional estudiado en clase.

15.2. Algoritmos

El módulo de ejercicios resueltos y el módulo de ejercicios interactivos se fundamentan directamente en el concepto de algoritmo como secuencia ordenada, finita y no ambigua de pasos orientados a la resolución de un problema. Cada ejercicio del sistema representa la implementación de un algoritmo aplicado a situaciones de la vida cotidiana, permitiendo al estudiante verificar que un algoritmo correctamente diseñado produce resultados consistentes y predecibles, independientemente del lenguaje de programación utilizado.

15.3. Pseudocódigo

El módulo de ejercicios resueltos incorpora la representación en pseudocódigo de cada algoritmo presentado. Esta sección permite al estudiante comprender la lógica de solución antes de observar la implementación formal en C++, reforzando el proceso de abstracción que caracteriza al diseño algorítmico. El uso del pseudocódigo como puente entre el razonamiento humano y el lenguaje de programación es uno de los ejes didácticos centrales del proyecto.

15.4. Diagramas de flujo

De manera complementaria al pseudocódigo, el módulo de ejercicios resueltos incluye diagramas de flujo que representan visualmente el flujo de control de cada algoritmo. El empleo de bloques terminales, bloques de proceso, bloques de entrada/salida y bloques condicionales permite que el estudiante comprenda la estructura lógica del problema antes de su codificación.

Esta representación refuerza la capacidad del usuario para identificar bifurcaciones, ciclos y secuencias dentro de un algoritmo.

15.5. Variables y operadores

En los ejercicios interactivos, el estudiante ingresa valores mediante formularios web que el sistema procesa como variables de entrada. Internamente, los programas desarrollados en C++ utilizan variables de distintos tipos de datos (enteros, flotantes, cadenas de caracteres) y operadores aritméticos, relacionales y lógicos para procesar dichas entradas y generar resultados. De esta forma, el estudiante puede observar en tiempo real la función que cumplen las variables y los operadores dentro de un programa funcional.

15.6. Estructuras de control

Los algoritmos codificados en C++ que conforman el núcleo lógico del aplicativo emplean estructuras de control secuenciales, condicionales (if, if-else, switch) e iterativas (for, while, do-while). La variedad de ejercicios presentados en la plataforma cubre cada una de estas estructuras, permitiendo al estudiante identificar su comportamiento y aplicarlos en la resolución de problemas reales, tales como el cálculo de promedios, la conversión de unidades o la validación de datos.

15.7. Funciones

El desarrollo del backend en Flask con Python y la lógica algorítmica en C++ hacen uso extensivo del concepto de función como unidad modular de código reutilizable. Los programas en C++ están organizados en funciones específicas que reciben parámetros, realizan un procesamiento determinado y retornan un resultado al backend. Este enfoque permite que el estudiante comprenda la importancia de la modularización en el diseño de software, así como el concepto de paso de parámetros y valor de retorno.

15.8. Arreglos y estructuras de datos

El sistema emplea arreglos y estructuras de datos en el procesamiento de conjuntos de información, como el almacenamiento temporal de respuestas del usuario, la gestión de listas de ejercicios y el manejo de historiales de intentos. En el módulo de reportes y estadísticas, la recuperación y presentación de datos tabulados desde la base de datos SQLite reproduce conceptualmente el manejo de arreglos y registros estudiado en la asignatura. Adicionalmente, los ejercicios prácticos del aplicativo pueden incluir problemas específicos sobre manipulación de arreglos en C++.

15.9. Programación orientada a objetos

Si bien la asignatura introduce la programación orientada a objetos como un paradigma de diseño, el proyecto materializa este concepto en su arquitectura. El backend desarrollado en Flask con Python organiza el sistema mediante clases y objetos que representan entidades del sistema como usuarios, ejercicios e intentos. De igual manera, los programas en C++ con mayor complejidad pueden estructurarse empleando clases, encapsulamiento y métodos, permitiendo que el estudiante reconozca la aplicabilidad de la POO en proyectos reales.

15.10. Resolución de problemas mediante programación

El eje central del proyecto es precisamente la resolución de problemas. Cada ejercicio interactivo de la plataforma está planteado a partir de situaciones cotidianas reconocibles por el estudiante como el cálculo de promedios estudiantiles, la conversión de temperaturas o la determinación de si un número es primo con el propósito de evidenciar que la programación es una herramienta de solución aplicable a la realidad. El módulo de retroalimentación complementa esta dimensión al orientar al estudiante sobre sus errores y guiarlo hacia la solución correcta.

15.11. Desarrollo de aplicaciones utilizando C++

C++ constituye el lenguaje de programación principal para la implementación de los algoritmos del sistema. Cada ejercicio resuelto y cada ejercicio interactivo tienen como núcleo

de procesamiento un programa desarrollado en C++, compilado y ejecutado desde el servidor mediante la integración con Flask. El uso de este lenguaje permite que el estudiante observe la implementación concreta de los algoritmos diseñados, reforzando las competencias de codificación adquiridas en la asignatura y evidenciando la capacidad de C++ para resolver problemas computacionales de diferente naturaleza y complejidad.

16. PLAN DE TRABAJO

El desarrollo del proyecto LogicWeb UTA Soluciones Reales se organiza en siete fases consecutivas que cubren desde el análisis de requerimientos hasta la presentación final del producto. Cada fase comprende un conjunto de actividades definidas orientadas a garantizar la calidad, funcionalidad y coherencia del sistema.

Fase 1 – Análisis

- Revisión y análisis del programa académico de la asignatura de Algoritmos y Lógica de Programación para identificar los temas prioritarios a cubrir.
- Levantamiento de requerimientos funcionales y no funcionales del sistema.
- Identificación de las necesidades de aprendizaje de los estudiantes de primer nivel de la carrera de Ingeniería en Software.
- Definición del alcance del proyecto, módulos principales y problemas de la vida real a incorporar como ejercicios.
- Selección y justificación de las tecnologías a utilizar: Flask, C++, HTML, CSS, Bootstrap, JavaScript y SQLite.

Fase 2 – Diseño

- Elaboración de los prototipos de interfaz de usuario en Figma, considerando principios de usabilidad y diseño intuitivo.
- Diseño de la arquitectura del sistema en tres capas: presentación, lógica de negocio y datos.

- Modelado de la base de datos: definición de entidades (usuarios, ejercicios, intentos, retroalimentación), atributos y relaciones.
- Diseño de los diagramas de flujo y pseudocódigos correspondientes a cada ejercicio planteado en la plataforma.
- Definición de la estructura de navegación del aplicativo web y su organización por módulos y unidades temáticas.

Fase 3 – Desarrollo

- Implementación del frontend con HTML5, CSS3, Bootstrap y JavaScript, siguiendo los prototipos aprobados en la fase de diseño.
- Desarrollo del backend con Flask (Python): configuración de rutas, controladores y servicios de la aplicación.
- Codificación de los ejercicios algorítmicos en C++ para cada tema de la asignatura, aplicados a problemas cotidianos.
- Integración de los programas en C++ con el backend de Flask mediante llamadas a subprocesos del sistema.
- Creación y configuración de la base de datos SQLite con las tablas, relaciones y restricciones definidas en el diseño.

Fase 4 – Integración

- Integración de los módulos desarrollados (frontend, backend, algoritmos en C++ y base de datos) en un sistema unificado y funcional.
- Configuración del flujo completo de datos: ingreso de datos del usuario procesamiento en C++ retorno de resultados almacenamiento en base de datos visualización en la interfaz.
- Verificación de la comunicación entre capas y resolución de conflictos de integración.

- Implementación de los módulos de retroalimentación y reportes de progreso con datos almacenados en SQLite.

Fase 5 – Pruebas

- Ejecución de pruebas funcionales sobre cada módulo del sistema para verificar el cumplimiento de los requerimientos definidos.
- Realización de pruebas de usabilidad con usuarios del grupo objetivo (estudiantes de primer nivel) para evaluar la experiencia de uso.
- Pruebas de validación de entradas: verificación del correcto manejo de datos incorrectos, campos vacíos y casos límite.
- Identificación y registro de errores o inconsistencias detectadas durante las pruebas.
- Corrección de fallas y ajustes de mejora en el sistema.

Fase 6 – Documentación

- Elaboración del informe técnico final del proyecto, incluyendo todos los apartados académicos requeridos.
- Redacción del manual de usuario con instrucciones claras para la navegación y uso del aplicativo.
- Elaboración de la guía técnica del sistema con descripción de la arquitectura, estructura de la base de datos y proceso de instalación.
- Revisión y edición final del documento del proyecto conforme a las normas de presentación académica establecidas.

Fase 7 – Presentación final

- Preparación del material de presentación (diapositivas) con los aspectos más relevantes del proyecto.
- Ensayo y preparación de la exposición oral ante el docente y compañeros.

- Demostración en vivo del funcionamiento del aplicativo web.
- Recepción de observaciones y retroalimentación por parte del docente evaluador.

17. CRONOGRAMA

La Tabla 1 presenta la planificación temporal del proyecto LogicWeb UTA – Soluciones Reales, distribuida en ocho semanas de trabajo. Las actividades han sido organizadas de forma progresiva, considerando las dependencias entre fases y la carga de trabajo estimada para el equipo de cuatro integrantes. Las fases contempladas son: análisis, diseño, desarrollo, integración, pruebas, documentación y presentación final.

Tabla 2

Cronograma de actividades para el desarrollo del proyecto LogicWeb UTA (8 semanas)

N ^o	Actividad	Sem. 1 Análisis	Sem. 2 Diseño	Sem. 3 Desarrollo	Sem. 4 Desarrollo	Sem. 5 Integración	Sem. 6 Pruebas	Sem. 7 Doc.	Sem. 8 Presentación
1	Análisis de requerimientos funcionales y no funcionales	✓							

2	Identificación de temas y ejercicios a incorporar	✓				
3	Selección y configuración del entorno de desarrollo	✓				
4	Diseño de prototipos de interfaz en Figma		✓			
5	Modelado de la base de datos SQLite		✓			
6	Diseño de diagramas de flujo y pseudocódigos		✓	✓		
7	Desarrollo del frontend (HTML, CSS, Bootstrap, JS)			✓	✓	
8	Desarrollo del backend con Flask (Python)			✓	✓	
9	Codificación de ejercicios algorítmicos en C++				✓	✓
10	Integración de C++ con el backend Flask					✓
11	Implementación de base de datos y módulo de reportes					✓
12	Integración completa de módulos del sistema				✓	✓
13	Pruebas funcionales y de usabilidad					✓
14	Corrección de errores y ajustes de mejora					✓

1	Elaboración		
5	del informe	✓	
	técnico final		
	Redacción del		
1	manual de	✓	
6	usuario y guía		
	técnica		
1	Preparación del		
7	material de		✓
	presentación		
1	Presentación y		
	demonstración		
8	final del		✓
	proyecto		

Nota. El símbolo ✓ indica las semanas en que cada actividad está planificada para su ejecución. Algunas actividades pueden solaparse con la semana siguiente según el avance del equipo de trabajo.

18. RECURSOS

Para el desarrollo, ejecución e implementación del proyecto "LogicWeb UTA", se contempla la utilización de recursos clasificados en cuatro categorías fundamentales: Humanos, Tecnológicos (Hardware y Software), e Institucionales.

18.1.Recursos Humanos

a. Equipo Desarrollador

Integrado por tres estudiantes de la carrera de Ingeniería en Software encargados del análisis, diseño de interfaces, codificación del backend/frontend, estructuración de la base de datos e integración de algoritmos.

b. Docente tutor

Profesional del área (Ing. José Rubén Caiza C.) responsable de la dirección académica, validación del marco teórico y supervisión metodológica del proyecto formativo.

c. Usuarios Beneficiarios

Estudiantes de primer semestre de Ingeniería en Software de la Universidad Técnica de Ambato (paralelo 1ero Software B) y terceros interesados que participarán en las fases de pruebas y evaluación del aplicativo.

18.2.Recursos tecnológicos en Hardware

a. Equipos de Cómputo

Tres ordenadores portátiles de gama media/alta (procesadores Intel Core i5/i7 o AMD Ryzen 5/7, mínimo 8 GB de Memoria RAM y almacenamiento en estado sólido) utilizados para la virtualización de servidores locales y compilación de código en C++

b. Infraestructura de Red

Conexión a internet de banda ancha (fibra óptica doméstica y conectividad universitaria) para la investigación documental, sincronización de repositorios y acceso a bibliotecas digitales.

18.3. Recursos tecnológicos en Software

a. Entornos y Herramientas de Desarrollo

Visual Studio Code como editor de código principal y Sqlite para la base de datos de nuestro programa.

b. Tecnologías del Backend

Lenguaje de programación Python 3.x utilizando el microframework Flask para la orquestación lógica del servidor web.

c. Tecnologías del Frontend

HTML5, CSS3, JavaScript y el framework de diseño Bootstrap para garantizar una interfaz responsiva y amigable.

d. Motor de Compilación

Compilador GCC/G++ para la ejecución nativa de los módulos algorítmicos desarrollados en C++.

e. Sistema de Gestión de Base de Datos

SQLite 3 para el almacenamiento ligero de usuarios, intentos y estadísticas de progreso académico.

f. Control de Versiones

Git y la plataforma GitHub para la colaboración y el despliegue del código fuente.

18.4. Recursos Institucionales

a. Espacios Físicos

Laboratorios de computación de la Facultad de Ingeniería en Sistemas, Electrónica e Industrial (FISEI) de la Universidad Técnica de Ambato. Hogar de cada uno de los colaboradores.

b. Fuentes de Información

Acceso al ecosistema de biblioteca digital y eLibro provisto por la institución para la fundamentación del marco teórico.

19. PRESUPUESTO ESTIMADO

Debido a que el proyecto se plantea como una propuesta académica de software libre y de carácter formativo integrador, el uso de software comercial y licenciamientos se reduce a \$0.00, apoyándose enteramente en tecnologías de código abierto y de uso gratuito.

A continuación, se detalla el presupuesto estimado considerando los costos de infraestructura propia, el valor referencial del talento humano y los gastos operativos devengados durante el ciclo de vida del desarrollo:

Tabla 3*Presupuesto estimado*

Categoría	Descripción	Cantidad	Costo Unitario	Costo Total	Fuente de Financiamiento
Hardware (Uso / Amortización)	Laptops de desarrollo	4	\$0	\$ 0	Recursos Propios (Estudiantes)
Software e Infraestructura	Licencias de IDEs y Entornos (VS Code, Figma)	--	\$ 0.00	\$ 0.00	Código Abierto / Plan Estudiantil
Software e Infraestructura	Frameworks y Motores (Flask, Python, C++, SQLite)	--	\$ 0.00	\$ 0.00	Licencia Libre (FOSS)
Servicios Conectividad	Servicio de Internet Fibra Óptica (2 meses de desarrollo)	2	\$ 30.00	\$ 60.00	Recursos Propios (Estudiantes)
Servicios Conectividad	Suministro Eléctrico (Consumo estimado por labor de cómputo)	2	\$ 10.00	\$ 20.00	Recursos Propios (Estudiantes)
Talento Humano (Referencial)	Desarrolladores de Software Junior (Horas académicas)	4	\$ 0.00 **	\$ 0.00	Inversión de Tiempo Académico
TOTAL ESTIMADO				\$ 80	Autofinanciado por el Equipo

Nota 1. El valor de hardware corresponde al costo estimado de depreciación y uso de los equipos tecnológicos del equipo durante el periodo académico, no a la compra de equipos nuevos.

Nota 2. El costo de desarrollo de software es \$0.00 por tratarse de un proyecto integrador obligatorio de la cátedra de Algoritmos de Programación, por lo que el esfuerzo se traduce en horas de dedicación estudiantil.

19.1. Viabilidad Financiera y Sostenibilidad

Como se justifica previamente en el documento, el uso exclusivo de herramientas como Flask, SQLite y C++ elimina barreras económicas. El proyecto demuestra ser altamente viable para la Universidad Técnica de Ambato puesto que no requiere de infraestructura en la nube costosa ni servidores dedicados corporativos en su fase inicial, pudiéndose desplegar de manera local en los laboratorios de la universidad o mediante servidores web gratuitos de pruebas académica.

20. RIESGOS DEL PROYECTO

La identificación y gestión de riesgos constituye un componente fundamental en la planificación de cualquier proyecto de desarrollo de software. Para el proyecto LogicWeb UTA – Soluciones Reales, se ha realizado un análisis de los principales riesgos que podrían afectar el cumplimiento de los objetivos, el cronograma o la calidad del producto final. Cada riesgo ha sido categorizado, valorado en términos de probabilidad e impacto, y acompañado de una estrategia de mitigación orientada a reducir su efecto sobre el proyecto.

La valoración de la probabilidad e impacto de cada riesgo se establece en tres niveles: Alta, Media y Baja, considerando las condiciones del equipo de trabajo, las tecnologías empleadas y el contexto académico en el que se desarrolla el proyecto.

Tabla 4

Matriz de riesgos del proyecto LogicWeb UTA

ID	Categoría	Descripción del Riesgo	Probabilidad	Impacto	Estrategia de Mitigación
R01	Técnico	Incompatibilidad entre la compilación de C++ y Flask al momento de la integración del backend con los módulos algorítmicos.	Alta	Alta	Realizar pruebas de integración tempranas y documentar el proceso de compilación mediante subprocesos del sistema.

R02	Técnico	Errores en la ejecución de programas C++ desde el servidor Flask por diferencias en el entorno de desarrollo y producción.	Media	Alta	Estandarizar el entorno de desarrollo mediante configuraciones compartidas en el repositorio GitHub.
R03	Planificación	Retraso en el cumplimiento de las fases del cronograma debido a la carga académica de los integrantes del equipo.	Alta	Media	Establecer hitos semanales y reuniones de seguimiento para redistribuir tareas oportunamente.
R04	Recursos	Pérdida o corrupción de código fuente por falta de respaldo adecuado del repositorio.	Baja	Alta	Uso obligatorio de Git con commits frecuentes y repositorio remoto en GitHub como respaldo principal.
R05	Funcional	Validación insuficiente de entradas del usuario que genere errores en la ejecución de los algoritmos en C++.	Media	Alta	Implementar validación en el frontend con JavaScript y en el backend con Flask antes de invocar los ejecutables.
R06	Usabilidad	Interfaz web con baja usabilidad o diseño no intuitivo que dificulte la navegación de los estudiantes.	Media	Media	Aplicar principios de diseño UX en los prototipos de Figma y realizar pruebas de usabilidad con usuarios del grupo objetivo.
R07	Académico	Contenido teórico o ejercicios planteados que no se alineen con el	Baja	Alta	Revisión y validación continua del contenido por parte del docente

		programa de la asignatura de Algoritmos y Lógica de Programación.			tutor durante el desarrollo del proyecto.
R08	Datos	Corrupción o pérdida de datos almacenados en la base de datos SQLite por manejo incorrecto de transacciones.	Baja	Media	Implementar manejo adecuado de excepciones en las operaciones de base de datos y realizar respaldos periódicos del archivo SQLite.

Nota. La probabilidad e impacto se valoran en tres niveles: Alta, Media y Baja. Los riesgos de impacto Alto requieren atención prioritaria durante el desarrollo.

20.1. Consideraciones generales sobre la gestión de riesgos

El equipo de desarrollo se compromete a revisar la matriz de riesgos de forma periódica durante cada fase del proyecto, actualizando la valoración de probabilidad e impacto conforme avance el desarrollo. Ante la materialización de cualquier riesgo identificado, se activará de manera inmediata la estrategia de mitigación correspondiente, con el fin de minimizar el impacto sobre el cronograma, la funcionalidad del sistema y la calidad académica del producto final.

Adicionalmente, el uso exclusivo de tecnologías de código abierto como Flask, C++, SQLite y Bootstrap reduce significativamente los riesgos asociados a licenciamiento y costos imprevistos, mientras que el control de versiones mediante Git y GitHub garantiza la trazabilidad del código fuente y la posibilidad de recuperación ante fallos técnicos.

21. RESULTADOS ESPERADOS

El proyecto LogicWeb UTA – Soluciones Reales tiene como propósito generar un conjunto de resultados concretos y verificables que evidencien el cumplimiento de los objetivos planteados y el aporte académico del sistema desarrollado. Los resultados esperados se definen en función de los módulos del aplicativo, las metas de aprendizaje de los estudiantes y la viabilidad técnica del proyecto, y constituyen los entregables que serán evaluados al finalizar el ciclo de desarrollo.

A continuación, se presenta la descripción detallada de cada resultado esperado, indicando su nivel de prioridad en el contexto del proyecto:

Tabla 5

Resultados esperados del proyecto LogicWeb UTA

ID	Resultado Esperado	Descripción	Prioridad
RE01	Aplicativo web funcional	Un aplicativo web completamente operativo que integre los módulos de gestión de usuarios, contenidos teóricos, ejercicios resueltos, ejercicios interactivos, retroalimentación y reportes de progreso.	Alta
RE02	Ejercicios interactivos en C++	Un conjunto de ejercicios prácticos desarrollados en C++ aplicados a problemas de la vida real, ejecutables desde la plataforma web mediante integración con Flask.	Alta
RE03	Módulo de retroalimentación activo	Un sistema de retroalimentación automática que analice las respuestas del estudiante, identifique errores y proporcione recomendaciones orientadas a reforzar el aprendizaje.	Alta
RE04	Base de datos operativa	Una base de datos SQLite correctamente estructurada con las entidades de usuarios, ejercicios e intentos, que permita el registro, almacenamiento y consulta del historial académico.	Alta
RE05	Interfaz de usuario intuitiva	Una interfaz web responsiva y amigable desarrollada con HTML5, CSS3, Bootstrap y JavaScript, que facilite la navegación del estudiante por los distintos módulos del sistema.	Media
RE06	Reportes de progreso académico	Un módulo de estadísticas que genere reportes visuales del desempeño del estudiante, identificando fortalezas y áreas de mejora a partir del historial de intentos almacenado.	Media
RE07	Documentación técnica completa	Un informe técnico final, manual de usuario y guía de instalación del sistema que permitan la	Media

		comprensión, replicación y mantenimiento futuro del aplicativo.	
RE08	Recurso de apoyo académico validado	Un recurso educativo validado mediante pruebas de usabilidad con estudiantes del primer nivel de Ingeniería en Software, que demuestre su efectividad como herramienta de refuerzo del aprendizaje.	Alta

Nota. Los resultados de prioridad Alta son indispensables para la entrega funcional del proyecto. Los de prioridad Media complementan la propuesta y fortalecen su valor académico.

21.1. Impacto académico esperado

Desde una perspectiva académica, el principal resultado esperado del proyecto es la mejora en la comprensión y aplicación práctica de los contenidos de la asignatura de Algoritmos y Lógica de Programación por parte de los estudiantes del primer nivel de la carrera de Ingeniería en Software de la Universidad Técnica de Ambato. Se espera que el aplicativo LogicWeb UTA contribuya a reducir la brecha existente entre la teoría impartida en clase y su aplicación en la resolución de problemas reales mediante el uso de C++.

Asimismo, se espera que el sistema sirva como recurso de apoyo complementario para los procesos de nivelación, refuerzo y autoaprendizaje, permitiendo que los estudiantes practiquen de manera autónoma fuera del horario de clases y reciban retroalimentación inmediata sobre su desempeño. El registro estadístico del progreso académico permitirá al docente identificar con mayor precisión las áreas de mayor dificultad dentro del grupo estudiantil, facilitando la toma de decisiones pedagógicas oportunas.

21.2. Indicadores de éxito

Para verificar el cumplimiento de los resultados esperados, se establecen los siguientes indicadores de éxito que serán evaluados durante la fase de pruebas y en la presentación final del proyecto:

- El aplicativo web ejecuta correctamente al menos el 90% de los ejercicios interactivos sin errores de ejecución ni fallos en la integración entre Flask y C++.
- La interfaz de usuario es calificada como intuitiva y fácil de navegar por al menos el 80% de los estudiantes participantes en las pruebas de usabilidad.
- El módulo de retroalimentación genera mensajes de orientación pertinentes en al menos el 85% de los casos de error registrados durante las pruebas funcionales.
- La base de datos SQLite almacena y recupera correctamente el historial de intentos y estadísticas de progreso sin pérdida de información en todos los escenarios de prueba.

- El informe técnico, el manual de usuario y la guía de instalación son entregados de manera completa y dentro del plazo académico establecido en el cronograma.

El cumplimiento de estos indicadores permitirá validar que el proyecto *LogicWeb UTA* ha alcanzado sus metas técnicas y académicas, constituyéndose como una herramienta de apoyo efectiva para la enseñanza de la programación en el contexto universitario.

Conclusiones

El proyecto *LogicWeb UTA* representa una propuesta adecuada para fortalecer el aprendizaje de Lógica y Algoritmos de Programación, ya que integra contenidos teóricos, ejercicios resueltos, actividades interactivas y retroalimentación en un solo entorno web educativo.

La propuesta responde a la necesidad de mejorar la relación entre la teoría y la práctica en los estudiantes de primeros niveles, facilitando una comprensión más clara de los temas mediante ejercicios aplicados a problemas de la vida real.

Además, el uso de tecnologías como Flask, C++, HTML, CSS, Bootstrap, JavaScript y SQLite permite desarrollar un sistema funcional, accesible y adecuado para fines académicos, con una estructura organizada y escalable.

Finalmente, el proyecto demuestra viabilidad técnica y económica, debido al uso de software libre y recursos tecnológicos de bajo costo, lo que favorece su posible implementación como herramienta de apoyo en la enseñanza universitaria.

Recomendaciones

Se recomienda realizar pruebas con estudiantes reales para evaluar la funcionalidad, usabilidad y efectividad del aplicativo, con el fin de identificar posibles mejoras antes de su implementación definitiva.

También se sugiere ampliar progresivamente el contenido del sistema, incorporando más ejercicios, temas y niveles de dificultad que permitan fortalecer el aprendizaje continuo de los usuarios.

Es importante mantener actualizado el material teórico, los ejercicios resueltos y la retroalimentación proporcionada por la plataforma, para garantizar que el aplicativo conserve su utilidad académica.

Por último, se recomienda considerar la implementación futura de *LogicWeb UTA* como recurso de apoyo institucional en clases, tutorías o procesos de nivelación académica dentro de la carrera.

Bibliografía

- Fritelli, A. V., & Guzman, J. (2020). *Algoritmos y estructuras de datos*. Obtenido de <https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/175249?page=5>
- Gaxiola Pcheco, C. G. (6 de Abril de 2008). *Metodología de la programación con pseudocódigo enfocado al lenguaje C*. Obtenido de <https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/75935?page=15>
- Moreno, J. P. (2015). *Programacion orientada a objetos*. Obtenido de <https://0x102wtab-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/106461?page=10>
- Trejos Buritica, O. I. (2017). *Logica de programacion*. Obtenido de <https://bibliotecadigital.utn.edu.ec/files/original/11059f3b97acc4779729d4d141b0993d02e26ccc.pdf>
- Universidad Europea. (9 de Octubre de 2023). *Apps educativas en Educación Infantil*. Obtenido de <https://universidadeuropea.com/blog/apps-educativas/#:~:text=El%20t%C3%A9rmino%20%22aplicaci%C3%B3n%20educativa%22%20engloba,apoyo%20en%20el%20contexto%20educativo>